

Name: Yashkumar Sonwane

Superset ID:7795993

create table Customers (
CustomerID int primary key,
Name varchar(100),
DOB date,
Balance decimal(12,2),
LastModified datetime,
IsVIP boolean default false
);

create table Accounts (
AccountID int primary key,
CustomerID int,
AccountType varchar(20),
Balance decimal(12,2),
LastModified datetime,
foreign key (CustomerID) references Customers(CustomerID)
);

create table Transactions (
TransactionID int primary key,
AccountID int,
TransactionDate datetime,
Amount decimal(12,2),
TransactionType varchar(10),
foreign key (AccountID) references Accounts(AccountID)
);

Name: Yashkumar Sonwane

Superset ID:7795993

create table Loans (
LoanID int primary key,
CustomerID int,
LoanAmount decimal(12,2),
InterestRate decimal(5,2),
StartDate date,
EndDate date,
foreign key (CustomerID) references Customers(CustomerID)
);

create table Employees (
EmployeeID int primary key,
Name varchar(100),
Position varchar(50),
Salary decimal(12,2),
Department varchar(50),
HireDate date
);

create table AuditLog (
LogID int auto_increment primary key,
LogMessage varchar(255),
LogDate datetime default current_timestamp
);

Name: Yashkumar Sonwane

Superset ID:7795993

insert into Customers (CustomerID, Name, DOB, Balance, LastModified, IsVIP) values
(1, 'Customer 1', '1960-05-15', 1000.00, now(), false),
(2, 'Customer 2', '1990-07-20', 15000.00, now(), false),
(3, 'Customer 3', '1955-02-10', 8000.00, now(), false),
(4, 'Customer 4', '2000-11-30', 500.00, now(), false);

insert into Accounts (AccountID, CustomerID, AccountType, Balance, LastModified) values
(1, 1, 'Savings', 1000.00, now()),
(2, 2, 'Checking', 15000.00, now()),
(3, 3, 'Savings', 8000.00, now()),
(4, 4, 'Checking', 500.00, now());

insert into Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType) values
(1, 1, now(), 200.00, 'Deposit'),
(2, 2, now(), 300.00, 'Withdrawal');

insert into Loans (LoanID, CustomerID, LoanAmount, InterestRate, StartDate, EndDate) values
(1, 1, 5000.00, 5.00, curdate(), date_add(curdate(), interval 60 month)),
(2, 3, 10000.00, 6.50, curdate(), date_add(curdate(), interval 15 day)),
(3, 2, 20000.00, 4.75, curdate(), date_add(curdate(), interval 20 day));

insert into Employees (EmployeeID, Name, Position, Salary, Department, HireDate) values
(1, 'Manager 1', 'Manager', 70000.00, 'HR', '2015-06-15'),
(2, 'Developer 1', 'Developer', 60000.00, 'IT', '2017-03-20');

Name: Yashkumar Sonwane

Superset ID:7795993

Exercise 1: Control Structures

Scenario 1

```
select l.LoanID, c.Name, c.DOB, l.InterestRate
from Loans l
join Customers c on l.CustomerID = c.CustomerID;
```

	LoanID	Name	DOB	InterestRate
▸	1	Customer 1	1960-05-15	5.00
	2	Customer 3	1955-02-10	6.50
	3	Customer 2	1990-07-20	4.75

```
delimiter $$

create procedure ApplySeniorLoanDiscount()
begin
  declare done int default false;
  declare customer_id int;
  declare date_of_birth date;
  declare age int;

  declare customer_cursor cursor for select CustomerID, DOB from Customers;
  declare continue handler for not found set done = true;

  open customer_cursor;

  read_loop: loop
```

Name: Yashkumar Sonwane

Superset ID:7795993

```
fetch customer_cursor into customer_id, date_of_birth;

if done then

    leave read_loop;

end if;

set age = timestampdiff(year, date_of_birth, curdate());

if age > 60 then

    update Loans

    set InterestRate = InterestRate - (InterestRate * 0.01)

    where CustomerID = customer_id;

end if;

end loop;

close customer_cursor;

end$$

delimiter ;

call ApplySeniorLoanDiscount();
```

```
select l.LoanID, c.Name, c.DOB, l.InterestRate

from Loans l

join Customers c on l.CustomerID = c.CustomerID;
```

	LoanID	Name	DOB	InterestRate
▸	1	Customer 1	1960-05-15	4.95
	2	Customer 3	1955-02-10	6.44
	3	Customer 2	1990-07-20	4.75

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 2

```
select CustomerID, Name, Balance, IsVIP from Customers;
```

	CustomerID	Name	Balance	IsVIP
▸	1	Customer 1	1000.00	0
	2	Customer 2	15000.00	0
	3	Customer 3	8000.00	0
	4	Customer 4	500.00	0

```
delimiter $$  
  
create procedure UpdateVIPStatus()  
begin  
    declare done int default false;  
    declare customer_id int;  
    declare customer_balance decimal(12,2);  
  
    declare customer_cursor cursor for select CustomerID, Balance from Customers;  
    declare continue handler for not found set done = true;  
  
    open customer_cursor;  
  
    read_loop: loop  
        fetch customer_cursor into customer_id, customer_balance;  
        if done then  
            leave read_loop;  
        end if;
```

Name: Yashkumar Sonwane

Superset ID:7795993

```

ifcustomer_balance > 10000 then
  update Customers set IsVIP = true where CustomerID = customer_id;
end if;
endloop;

closecustomer_cursor;
end$$

delimiter ;

callUpdateVIPStatus();

```

```

selectCustomerID, Name, Balance, IsVIP from Customers;

```

	CustomerID	Name	Balance	IsVIP
▸	1	Customer 1	1000.00	0
	2	Customer 2	15000.00	1
	3	Customer 3	8000.00	0
	4	Customer 4	500.00	0

Scenario 3

```
select LoanID, CustomerID, EndDate from Loans;
```

	LoanID	CustomerID	EndDate
▸	1	1	2031-07-03
	2	3	2026-07-18
	3	2	2026-07-23

```
delimiter $$

create procedure SendLoanDueReminders()
begin
  declare done int default false;
  declare loan_id int;
  declare customer_id int;
  declare due_date date;
  declare customer_name varchar(100);

  declare loan_cursor cursor for
    select LoanID, CustomerID, EndDate from Loans
    where EndDate between curdate() and date_add(curdate(), interval 30 day);
  declare continue handler for not found set done = true;

  open loan_cursor;

  read_loop: loop
    fetch loan_cursor into loan_id, customer_id, due_date;
```


Name:KalyanRanjan

SupersetID:7795995

ifdonethen
leaveread_loop;
end if;
selectName into customer_name from Customers where CustomerID = customer_id;
selectconcat(customer_name, ', Your Loan ', loan_id, ' Is Due On ', due_date) as Reminder;
endloop;
closeloan_cursor;
end\$\$
delimiter ;
callSendLoanDueReminders();

	Reminder
▸	Customer 2, Your Loan 3 Is Due On 2026-07-23

Exercise 2: Error Handling

Scenario 1

```
select AccountID, Balance from Accounts where AccountID in (1, 2, 4);
```

	AccountID	Balance
▸	1	1000.00
	2	15000.00
	4	500.00

```
delimiter $$  
  
create procedure SafeTransferFunds(  
  in from_account_id int,  
  in to_account_id int,  
  in transfer_amount decimal(12,2)  
)  
begin  
  declare source_balance decimal(12,2);  
  
  declare exit handler for sqlexception  
  begin  
    rollback;  
    insert into AuditLog (LogMessage)  
    values (concat('Error: Transfer From Account ', from_account_id, ' Failed'));  
  end;
```

Name: Yashkumar Sonwane

Superset ID:7795993

start transaction;
select Balance into source_balance from Accounts where AccountID = from_account_id;
ifsource_balance < transfer_amount then
signal sqlstate '45000' set message_text = 'Insufficient Funds';
end if;
update Accounts set Balance = Balance - transfer_amount where AccountID = from_account_id;
update Accounts set Balance = Balance + transfer_amount where AccountID = to_account_id;
commit;
end\$\$
delimiter ;
callSafeTransferFunds(2, 1, 100.00);
callSafeTransferFunds(4, 1, 999999.00);

select AccountID, Balance from Accounts where AccountID in (1, 2, 4);

	AccountID	Balance
▸	1	1100.00
	2	14900.00
	4	500.00
•	NULL	NULL

Name: Yashkumar Sonwane

Superset ID:7795993

select *from AuditLog;

	LogID	LogMessage	LogDate
▸	1	Error: Transfer From Account 4 Failed	2026-07-03 05:04:02

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 2

```
select EmployeeID, Name, Salary from Employees;
```

	EmployeeID	Name	Salary
▸	1	Manager 1	70000.00
	2	Developer 1	60000.00

```
delimiter $$  
  
create procedure UpdateSalary(  
  in employee_id int,  
  in increase_percentage decimal(5,2)  
)  
begin  
  declare employee_count int;  
  
  select count(*) into employee_count from Employees where EmployeeID = employee_id;  
  
  if employee_count = 0 then  
    insert into AuditLog (LogMessage)  
    values (concat('Error: Employee Id ', employee_id, ' Does Not Exist'));  
  else  
    update Employees  
    set Salary = Salary + (Salary * increase_percentage / 100)  
    where EmployeeID = employee_id;  
  end if;  
end$$
```

Name: Yashkumar Sonwane

Superset ID:7795993

delimiter ;

call UpdateSalary(1, 10.00);

call UpdateSalary(999, 10.00);

select EmployeeID, Name, Salary from Employees;

	EmployeeID	Name	Salary
▸	1	Manager 1	77000.00
	2	Developer 1	60000.00

select *from AuditLog;

	LogID	LogMessage	LogDate
▸	1	Error: Transfer From Account 4 Failed	2026-07-03 05:04:02
	2	Error: Employee Id 999 Does Not Exist	2026-07-03 05:13:23

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 3

```
select CustomerID, Name from Customers;
```

	CustomerID	Name
▸	1	Customer 1
	2	Customer 2
	3	Customer 3
	4	Customer 4

```
delimiter $$

create procedure AddNewCustomer(
  in customer_id int,
  in customer_name varchar(100),
  in date_of_birth date,
  in balance decimal(12,2)
)
begin
  declare existing_count int;

  select count(*) into existing_count from Customers where CustomerID = customer_id;

  if existing_count > 0 then
    insert into AuditLog (LogMessage)
    values (concat('Error: Customer Id ', customer_id, ' Already Exists'));
  else
    insert into Customers (CustomerID, Name, DOB, Balance, LastModified)
```

Name: Yashkumar Sonwane

Superset ID:7795993

```
values (customer_id, customer_name, date_of_birth, balance, now());  
  
end if;  
  
end$$  
  
  
delimiter ;  
  
  
call AddNewCustomer(5, 'Customer 5', '1995-04-12', 2000.00);  
call AddNewCustomer(1, 'New Customer 1', '1985-05-15', 1000.00);
```

```
select CustomerID, Name from Customers;
```

	CustomerID	Name
▸	1	Customer 1
	2	Customer 2
	3	Customer 3
	4	Customer 4
	5	Customer 5

```
select *from AuditLog;
```

	LogID	LogMessage	LogDate
▸	1	Error: Transfer From Account 4 Failed	2026-07-03 05:04:02
	2	Error: Employee Id 999 Does Not Exist	2026-07-03 05:13:23
	3	Error: Customer Id 1 Already Exists	2026-07-03 05:27:31

Exercise 3: Stored Procedures

Scenario 1

```
select AccountID, AccountType, Balance from Accounts where AccountType = 'Savings';
```

	AccountID	AccountType	Balance
▸	1	Savings	1100.00
	3	Savings	8000.00

```
delimiter $$
```

```
create procedure ProcessMonthlyInterest()
```

```
begin
```

```
  update Accounts
```

```
    set Balance = Balance + (Balance * 0.01)
```

```
  where AccountType = 'Savings';
```

```
end$$
```

```
delimiter ;
```

```
call ProcessMonthlyInterest();
```

```
select AccountID, AccountType, Balance from Accounts where AccountType = 'Savings';
```

	AccountID	AccountType	Balance
▸	1	Savings	1111.00
	3	Savings	8080.00

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 2

```
select EmployeeID, Name, Department, Salary from Employees where Department = 'IT';
```

	EmployeeID	Name	Department	Salary
▸	2	Developer 1	IT	60000.00

```
delimiter $$  
  
create procedure UpdateEmployeeBonus(  
    in department_name varchar(50),  
    in bonus_percentage decimal(5,2)  
)  
begin  
    update Employees  
    set Salary = Salary + (Salary * bonus_percentage / 100)  
    where Department = department_name;  
end$$  
  
delimiter ;  
  
call UpdateEmployeeBonus('IT', 5.00);
```

```
select EmployeeID, Name, Department, Salary from Employees where Department = 'IT';
```

	EmployeeID	Name	Department	Salary
▸	2	Developer 1	IT	63000.00

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 3

```
select AccountID, Balance from Accounts where AccountID in (1, 2);
```

	AccountID	Balance
▸	1	1111.00
	2	14900.00

```
delimiter $$  
  
create procedure TransferFunds(  
  infrom_account_id int,  
  into_account_id int,  
  intransfer_amount decimal(12,2)  
)  
begin  
  declare source_balance decimal(12,2);  
  
  select Balance into source_balance from Accounts where AccountID = from_account_id;  
  
  ifsource_balance >= transfer_amount then  
    update Accounts set Balance = Balance - transfer_amount where AccountID = from_account_id;  
    update Accounts set Balance = Balance + transfer_amount where AccountID = to_account_id;  
  end if;  
end$$  
  
delimiter ;
```

Name: Yashkumar Sonwane

Superset ID:7795993

```
call TransferFunds(2, 1, 500.00);
```

```
select AccountID, Balance from Accounts where AccountID in (1, 2);
```

	AccountID	Balance
▸	1	1611.00
	2	14400.00

Exercise 4:Functions

Scenario 1

delimiter \$\$
create function CalculateAge(date_of_birth date)
returns int
deterministic
begin
return timestampdiff(year, date_of_birth, curdate());
end\$\$
delimiter ;

select CustomerID, Name, DOB, CalculateAge(DOB) as Age from Customers;
--

	CustomerID	Name	DOB	Age
▸	1	Customer 1	1960-05-15	66
	2	Customer 2	1990-07-20	35
	3	Customer 3	1955-02-10	71
	4	Customer 4	2000-11-30	25
	5	Customer 5	1995-04-12	31

Scenario 2

delimiter \$\$
create function CalculateMonthlyInstallment(
loan_amount decimal(12,2),
interest_rate decimal(5,2),
duration_years int
)
returns decimal(12,2)
deterministic
begin
declare monthly_rate decimal(10,6);
declare number_of_payments int;
set monthly_rate = (interest_rate / 100) / 12;
set number_of_payments = duration_years * 12;
return round((loan_amount * monthly_rate) / (1 - power(1 + monthly_rate, -number_of_payments)), 2);
end\$\$
delimiter ;

select LoanID, LoanAmount, InterestRate,
CalculateMonthlyInstallment(LoanAmount, InterestRate, 5) as MonthlyInstallment
from Loans;

Name: Yashkumar Sonwane

Superset ID:7795993

	LoanID	LoanAmount	InterestRate	MonthlyInstallment
▸	1	5000.00	4.95	94.24
	2	10000.00	6.44	195.38
	3	20000.00	4.75	375.13

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 3

delimiter \$\$
create function HasSufficientBalance(account_id int, amount decimal(12,2))
returns boolean
deterministic
begin
declare account_balance decimal(12,2);
select Balance into account_balance from Accounts where AccountID = account_id;
return account_balance >= amount;
end\$\$
delimiter ;

select AccountID, Balance, HasSufficientBalance(AccountID, 500) as HasFiveHundred from Accounts;
--

	AccountID	Balance	HasFiveHundred
▸	1	1611.00	1
	2	14400.00	1
	3	8080.00	1
	4	500.00	1

Exercise 5: Triggers

Scenario 1

```
delimiter $$  
  
create trigger UpdateCustomerLastModified  
before update on Customers  
for each row  
begin  
    set new.LastModified = now();  
end$$  
  
delimiter ;
```

```
selectCustomerID, Balance, LastModified from Customers where CustomerID = 1;
```

	CustomerID	Balance	LastModified
▸	1	1000.00	2026-07-03 04:21:03

```
updateCustomers set Balance = Balance + 50 where CustomerID = 1;  
selectCustomerID, Balance, LastModified from Customers where CustomerID = 1;
```

	CustomerID	Balance	LastModified
▸	1	1050.00	2026-07-03 06:10:07

Scenario 2

```
delimiter $$  
  
create trigger LogTransaction  
after insert on Transactions  
for each row  
begin  
    insert into AuditLog (LogMessage)  
    values (concat(new.TransactionType, ' Of ', new.Amount, ' On Account ', new.AccountID));  
end$$  
  
delimiter ;
```

```
select*from AuditLog;
```

	LogID	LogMessage	LogDate
▸	1	Error: Transfer From Account 4 Failed	2026-07-03 05:04:02
	2	Error: Employee Id 999 Does Not Exist	2026-07-03 05:13:23
	3	Error: Customer Id 1 Already Exists	2026-07-03 05:27:31

```
insertinto Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)  
values(3, 3, now(), 750.00, 'Deposit');  
select*from AuditLog;
```

Name: Yashkumar Sonwane

Superset ID:7795993

	LogID	LogMessage	LogDate
▸	1	Error: Transfer From Account 4 Failed	2026-07-03 05:04:02
	2	Error: Employee Id 999 Does Not Exist	2026-07-03 05:13:23
	3	Error: Customer Id 1 Already Exists	2026-07-03 05:27:31
	4	Deposit Of 750.00 On Account 3	2026-07-03 06:16:52

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 3

delimiter \$\$
create trigger CheckTransactionRules
before insert on Transactions
for each row
begin
declare account_balance decimal(12,2);
select Balance into account_balance from Accounts where AccountID = new.AccountID;
if new.TransactionType = 'Withdrawal' and new.Amount > account_balance then
signal sqlstate '45000' set message_text = 'Withdrawal Exceeds Balance';
end if;
if new.TransactionType = 'Deposit' and new.Amount <= 0 then
signal sqlstate '45000' set message_text = 'Deposit Must Be Positive';
end if;
end\$\$
delimiter ;

select *from Transactions;

Name: Yashkumar Sonwane

Superset ID:7795993

	TransactionID	AccountID	TransactionDate	Amount	TransactionType
▸	1	1	2026-07-03 04:21:03	200.00	Deposit
	2	2	2026-07-03 04:21:03	300.00	Withdrawal
	3	3	2026-07-03 06:16:52	750.00	Deposit

```
insertintoTransactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
values(4,1, now(), 100.00, 'Withdrawal');
select*from Transactions;
```

	TransactionID	AccountID	TransactionDate	Amount	TransactionType
▸	1	1	2026-07-03 04:21:03	200.00	Deposit
	2	2	2026-07-03 04:21:03	300.00	Withdrawal
	3	3	2026-07-03 06:16:52	750.00	Deposit
	4	1	2026-07-03 06:22:20	100.00	Withdrawal

Name: Yashkumar Sonwane

Superset ID:7795993

Exercise 6: Cursors

Scenario 1

delimiter \$\$
create procedure GenerateMonthlyStatements()
begin
declare done int default false;
declare customer_name varchar(100);
declare account_id int;
declare transaction_date datetime;
declare amount decimal(12,2);
declare transaction_type varchar(10);
declare transaction_cursor cursor for
select c.Name, t.AccountID, t.TransactionDate, t.Amount, t.TransactionType
from Transactions t
join Accounts a on t.AccountID = a.AccountID
join Customers c on a.CustomerID = c.CustomerID
where month(t.TransactionDate) = month(curdate())
and year(t.TransactionDate) = year(curdate());
declare continue handler for not found set done = true;
open transaction_cursor;
read_loop: loop
fetch transaction_cursor into customer_name, account_id, transaction_date, amount, transaction_type;
if done then

Name: Yashkumar Sonwane

Superset ID:7795993

leaveread_loop;
end if;
selectconcat('Statement - ', customer_name, ', Account ', account_id, ', ', transaction_type, ' Of ', amount,'On ', transaction_date) as Statement;
endloop;
closetransaction_cursor;
end\$\$
delimiter ;
callGenerateMonthlyStatements();

	Statement
▸	Statement - Customer 1, Account 1, Withdrawal Of 100.00 On 2026-07-03 06:22:20

Name: Yashkumar Sonwane

Superset ID:7795993

Scenario 2

```
select AccountID, Balance from Accounts;
```

	AccountID	Balance
▸	1	1611.00
	2	14400.00
	3	8080.00
	4	500.00

```
delimiter $$
```

```
create procedure ApplyAnnualFee(in fee_amount decimal(12,2))
```

```
begin
```

```
    declare done int default false;
```

```
    declare account_id int;
```

```
    declare account_cursor cursor for select AccountID from Accounts;
```

```
    declare continue handler for not found set done = true;
```

```
    open account_cursor;
```

```
    read_loop: loop
```

```
        fetch account_cursor into account_id;
```

```
        if done then
```

```
            leave read_loop;
```


Name: Yashkumar Sonwane

Superset ID:7795993

update Accounts set Balance = Balance - fee_amount where AccountID = account_id;
end loop;
close account_cursor;
end\$\$
delimiter ;
call ApplyAnnualFee(25.00);

select AccountID, Balance from Accounts;
--

	AccountID	Balance
▸	1	1586.00
	2	14375.00
	3	8055.00
	4	475.00

Scenario 3

```
select LoanID, InterestRate from Loans;
```

	LoanID	InterestRate
▸	1	4.95
	2	6.44
	3	4.75

```
delimiter $$  
  
create procedure UpdateLoanInterestRates(in rate_increase decimal(5,2))  
begin  
    declare done int default false;  
    declare loan_id int;  
  
    declare loan_cursor cursor for select LoanID from Loans;  
    declare continue handler for not found set done = true;  
  
    open loan_cursor;  
  
    read_loop: loop  
        fetch loan_cursor into loan_id;  
        if done then  
            leave read_loop;
```

Name: Yashkumar Sonwane

Superset ID:7795993

end if;
update Loans set InterestRate = InterestRate + rate_increase where LoanID = loan_id;
end loop;
close loan_cursor;
end\$\$
delimiter ;
call UpdateLoanInterestRates(0.50);

select LoanID, InterestRate from Loans;

	LoanID	InterestRate
▸	1	5.45
	2	6.94
	3	5.25